

CMOR 646 Final Report

Devin Abraham and Ben Freidinger

May 2026

1 Introduction

The k-server problem is one of the most influential online problems with applications to paging, caching, and resource allocation. In this problem, k servers move within a metric space to serve sequential online requests, with the goal of minimizing total distance traveled. Manasse, McGeoch, and Sleator [1] presented the first competitive algorithms for this problem, including the BAL algorithm which assigns requests to the server with least total distance traveled. The k-taxi variant extends this model by requiring servers to transport requests from a source to a destination, rather than simply visiting the request location. This variant directly models applications like ride-sharing and elevator scheduling, where pickup and delivery are both essential components of service.

Recent work has extended the k-server and k-taxi problems in several directions. Dehghani et al. [2] analyzed the stochastic k-server problem where requests are drawn from known time-dependent distributions, achieving a 3-approximation for line and circle metrics. They extended their results to the "Uber problem" (k-taxi with pickup and delivery), finding α approximations for particular k values. Coester and Koutsoupias [3] proved the first competitive algorithms for the online k-taxi problem in general metrics, while recent work by Coester and Poon [4] achieved constant competitive ratios for $k=3$ taxis. However, all existing work assumes unlimited server capacity—a critical limitation for real-world applications like elevator systems where strict capacity constraints (typically 8-16 passengers) fundamentally change the problem structure.

We propose to analyze the capacity-constrained k-taxi problem on the line metric (for simplicity), modeling elevator scheduling with k elevators each having capacity C . We will formulate the offline optimal solution as an integer program based on the model devised by Dehghani et. al. with capacity constraints added. We will then analyze solutions to this model and attempt to use the property that on the line metric, a fractional online algorithm has an online integral solution for the same problem with the same expected cost. We will also explore algorithms such as **REGIONTRACKER** proposed by Coester and Koutsoupias with modifications to account for capacity constraints. Finally, we propose a two stage stochastic LP to model the elevator problem and use MPC to solve an approximation of the formulation.

2 Literature Review

"The Online k-taxi Problem" by Coester and Koutsoupias [3]

This is a foundational paper in the study of the k-taxi problem. The authors begin by defining the problem and differentiating the two main forms: the easy k-taxi problem where we consider minimizing the total distance traveled by all taxis and the hard k-taxi problem where we only consider the distance traveled by empty taxis (deadheading). A couple important notational things from this paper:

- $A(C, \sigma)$ - run of algorithm A with initial configuration C and arrivals σ
- $cost_A(C, \sigma)$ - the cost of algorithm A on initial configuration C with arrivals σ

The paper then discusses an algorithm for the 2-taxi case, **BIASEDDC**. That labels the two taxis according to whether they fulfilled the last request or not: the one that filled the last request is labeled A for active and the one that did not is labeled P for passive. When the next request arrives, the algorithm sends both taxis to the location of the request, but sends P at twice the speed of A . They state and provide a proof sketch that this algorithm is 9-competitive for the hard 2-taxi problem. They then describe an algorithm, **REGIONTRACKER**, for the hard 3-taxi problem on the line metric. This algorithm sets intervals around each taxi to attempt to maintain some distance. It then determines how fast to move each taxi using these regions and simultaneously updates region boundaries while serving each request. They use a potential to show that this algorithm achieves a constant competitive ratio. They also devote sections to the discussion of the k -taxi problem on more general metrics such as hierarchically separate trees (HSTs) and connections to the layered graph transversal problem, which are not relevant to our formulation, so we will not include that information. In their conclusion the authors list the following topics as areas for continued research:

- Finding an algorithm for the hard k -taxi problem on general metric spaces with competitive ratio based only on k
- Using the **REGIONTRACKER** algorithm as a basis for other algorithms on different metric spaces (or potentially as a basis for an algorithm for the capacity-constrained version)
- Other results specific to HSTs

”Decision Theoretic Group Elevator Schedule” by Daniel Nikovski and Matthew Brand [5]

The authors of this paper formulate the elevator problem as a discrete state Markov chain with the goal of minimizing expected passenger waiting time for 2-8 elevators. The paper makes the assumptions of the destination floors of passengers are equally likely and the number of people standing on each floor is known, and assumes that each car has infinite capacity. The authors consider the speed and acceleration of the elevator to create branching points, given the elevators current velocity and direction what is the closet floor it can stop at. The algorithm employs dynamic programming to solve in $O(N_f N_p)$ where N_f, N_p are the number of floors and number of passengers respectively. Their Markov Chain has a 4 variable state space of (f, d, v, n) where f is the floor of the car, d is the current direction, v is its current velocity, and n is the number of newly boarded passengers. Their solution had 30% – 40% gains when compared to other state of the art algorithms at the time indicating a great success for the MDP approach. They noted a possible extension of their work is to model the problem with capacity constraints. The authors suggest setting the capacity to C , and the maximum number of people that would enter the car as H where $C < H$. Then given a velocity divide the $H + 1$ states into two subgroups of $C + 1$ which is the number of people let onto the car, and $H - m + 1$ states which correspond to the possible number of people left off as a result of a car overflow.

”Stochastic k -Server: How Should Uber Work?” by Dehghani et al. [2]

This paper introduces the Stochastic k -Server problem where instead of the classic adversarial approach the job requests are drawn from a given probability distribution allowing algorithms to take advantage of demand patterns. The authors provide an exponential size dynamic programming formulation to achieve the optimal strategy, but in practice it is too computationally intensive to solve. In addition, the authors define the Uber problem as an extension of the k -Server problem where the server work demand consists of a source and destination like Uber and elevator formulations. They also provide a Integer Program and LP relaxation to solve the k -Server problem which we attach below. They prove for any online algorithm, there exists a request oblivious algorithm that is at worst 3 times the cost of the online algorithm. This means that for any online algorithm there is one that sets the servers to a prespecified configuration before time i servers the request at time i then goes to a new prespecified

location. An important result is when working in the line metric (which is the elevator problem) a fractional online algorithm has an online integral solution for the same problem with the same expected result which means any fractional solution found by the LP relaxation will have an integral solution that will be practically done.

”Stochastic Vehicle Routing with Recourse” by Gørtz et al. [6]

The authors model this problem as a two stage stochastic recourse framework as we intend to model the elevator routing problem as. They compute a route without knowing the true demand, then fixing a recourse route to fix mistakes after knowing the true demand. An important idea from this paper is using a recourse route factor $\lambda > 1$. This gives the idea of urgency and creating the best initial route which separates the first route from the second route since the second route will be penalized from the $\lambda > 1$. The second stage actions are more expensive than the first stage actions which models the urgency that vehicles or elevators have in the second stage. If $\lambda = 1$, then the optimal policy would be to do everything in the recourse stage once the demand is realized. In addition, the authors reduce the StochVRP problem to a set cover problem, and solve with a greedy set cover approximation algorithm. Then to evaluate how good a route is the authors use submodular orienteering to approximate the objective function.

”Fast Model Predictive Control Using Online Optimization” by Wang et al. [7]

Model Predictive Control (MPC) is a control framework in which a finite-horizon optimization problem is solved at each time step using a model of the system to predict future behavior. The optimization explicitly incorporates system dynamics, control constraints, and an objective function that captures performance goals such as minimizing travel distance or delay. After solving this problem, only the first control action is implemented, and the process is repeated at the next time step with updated system information. This receding-horizon structure provides a form of feedback, allowing the controller to adapt to changes and disturbances in real time while still planning ahead. As emphasized by Wang and Boyd, the effectiveness of MPC depends on solving these optimization problems efficiently online, making computational tractability a central consideration in its design and application.

3 k-Taxi Formulation

We present a formulation based heavily on the model from the Dehghani et. al. paper, but changed slightly to account for taxis instead of servers. The authors model this problem as a two stage stochastic recourse framework, the same type of model we intend to use for the elevator routing problem. They compute a route without knowing the true demand and then fix a recourse route to fix errors after knowing the true demand. We combine their model with the recourse route factor $\lambda > 1$ [6]. If $\lambda = 1$, then the optimal policy would be to move all elevators in the recourse stage once the demand is observed. When $\lambda > 1$, there is an incentive to optimize movements in the first stage to minimize unnecessary movements but also to set up well for any possible payout of the second stage. This has a nice real world interpretation as it implicitly helps reduce passenger wait times.

Initial k-Taxi Formulation

$$\begin{aligned}
\min. \quad & \sum_{\tau} \sum_{u,v} f_{\tau,u,v} d(u,v) + \lambda \sum_{\tau} \sum_v \sum_r x_{\tau,v,r} \Pr(z \sim P_{\tau} = r) [d(v, s_r) + d(s_r, d_r)] \\
\forall \tau, v \quad & b_{\tau+1,v} = b_{\tau,v} + \sum_u f_{\tau,u,v} - \sum_u f_{\tau,v,u} - \sum_r x_{\tau,v,r} + \sum_r \sum_u x_{\tau,u,r} \mathbf{1}[d_r = v]. \\
\forall \tau, u, v \quad & \sum_v x_{\tau,v,r} \geq 1. \\
\forall \tau, v, r \quad & x_{\tau,v,r} \leq b_{\tau,v}. \\
\forall \tau \quad & \sum_v b_{\tau,v} \leq k. \\
\forall \tau, v, r \quad & x_{\tau,v,r} \in \{0, 1\}. \\
\forall \tau, u, v \quad & f_{\tau,u,v} \in \mathbb{N}. \\
\forall \tau, v \quad & b_{\tau,v} \in \mathbb{N}.
\end{aligned}$$

Variable Definitions

- τ A time step in the sequence of configurations.
- v, u Nodes in the metric space (line).
- r A possible request, defined by a pickup location s_r and a dropoff location d_r .
- k The total number of taxis.
- $b_{\tau,v}$ The number of taxis sitting idle at node v at time τ .
- $f_{\tau,u,v}$ The number of taxis that reposition (move empty) from node u to node v at time τ (voluntary repositioning).
- $x_{\tau,v,r}$ A binary variable indicating whether a taxi at node v is assigned to serve request r at time τ .
- $d(u, v)$ The distance between nodes u and v in the metric space.
- $\Pr(z \sim P_{\tau} = r)$ The probability that request r is the one that arrives at time τ .

Note that $f_{\tau,u,v}$ and $x_{\tau,v,r}$ are different: f captures voluntary empty repositioning (proactive movement of taxis between nodes), while x captures the assignment of a taxi to serve a request, which triggers the mandatory trip from s_r to d_r .

4 Two-Stage Stochastic k -Taxi Formulation with Capacity Constraints

We extend the base k -taxi IP to handle capacity constraints with the same two-stage stochastic format. The information structure at each time step τ is as follows.

1. **Request arrival.** A request r is drawn from P_{τ} , revealing the pickup location s_r and destination d_r . The group size n_r is not yet known.

2. **Stage 1 (here-and-now).** Knowing r , we reposition taxis toward s_r . At least one taxi must arrive at s_r .
3. **Information event.** The first dispatched taxi arrives at s_r and the group size n_r is revealed.
4. **Stage 2 (recourse).** Given the realized group size, we assign taxis to serve the request. Each assigned taxi travels from its current position to s_r and then to d_r .

Because Stage 1 decisions are made after observing r , the repositioning variables carry an r index.

Model Simplifications

1. **No recourse repositioning** - it would never be optimal to pay the $\lambda > 1$ fee to reposition in stage 2 of time τ when you could instead wait until stage 1 of time $\tau + 1$ with more information available.
2. **Scenario collapse** - we do not need to index each stage 2 scenario by the size of the demand but rather by the number of taxis required to fulfill that demand. So we index the scenarios as follows:

$$m \in M = \{1, 2, \dots, \lceil H/C \rceil\}.$$

. And use the following probability for each scenario m :

$$q_m = \Pr(\lceil n_r/C \rceil = m) = \sum_{\ell=(m-1)C+1}^{\min(mC, H)} \Pr(n_r = \ell), \quad \sum_{m \in M} q_m = 1.$$

Additional Notation

We keep all notation from the base k -taxi formulation and introduce the following:

- C : uniform capacity of each taxi.
- H : maximum possible group size, with $H > C$.
- $M = \{1, \dots, \lceil H/C \rceil\}$: set of collapsed scenarios, indexed by number of taxis required.
- q_m : probability that scenario m occurs, as defined above.
- $\lambda > 1$: recourse penalty. Following Gørtz and Nagarajan, this scalar multiplicatively scales all Stage 2 costs relative to Stage 1
- $\tilde{b}_{\tau,v,r}$: number of idle taxis at node v at time τ after Stage 1 repositioning for request r , but before any Stage 2 assignment.

Decision Variables

Stage 1.

- $b_{\tau,v} \in \mathbb{N}$: number of idle taxis at node v at the *start* of time step τ , before the request is drawn. These are inherited from the previous time step.
- $f_{\tau,u,v,r} \in \mathbb{N}$: the number of taxis repositioned from u to v at time τ in response to request r . These are chosen *after* observing r but *before* the group size is revealed.

Stage 2.

- $y_{\tau,v,r,m} \in \mathbb{Z}_{\geq 0}$: the number of taxis at node v assigned to serve request r under scenario m . Each assigned taxi travels from v to s_r (deadhead) and then from s_r to d_r .

Formulation

$$\min \sum_{\tau} \sum_r \Pr(z \sim P_{\tau} = r) \left[\underbrace{\sum_{u,v} f_{\tau,u,v,r} d(u,v)}_{\text{Stage 1 repositioning}} + \lambda \sum_{m \in M} q_m \underbrace{\sum_v y_{\tau,v,r,m} (d(v, s_r) + d(s_r, d_r))}_{\text{Stage 2 assignment}} \right] \quad (\text{OBJ})$$

Stage 1 constraints.

$$\tilde{b}_{\tau,v,r} = b_{\tau,v} + \sum_u f_{\tau,u,v,r} - \sum_u f_{\tau,v,u,r} \quad \forall \tau, v, r \quad (\text{S1.1})$$

$$\sum_v b_{\tau,v} \leq k \quad \forall \tau \quad (\text{S1.2})$$

$$\tilde{b}_{\tau,s_r,r} \geq 1 \quad \forall \tau, r : \Pr(z \sim P_{\tau} = r) > 0 \quad (\text{S1.3})$$

Stage 2 constraints (for each collapsed scenario $m \in M$).

$$\sum_v y_{\tau,v,r,m} \geq m \quad \forall \tau, r, m \quad (\text{S2.1})$$

$$y_{\tau,v,r,m} \leq \tilde{b}_{\tau,v,r} \quad \forall \tau, v, r, m \quad (\text{S2.2})$$

Flow balance:

$$b_{\tau+1,v} = \sum_r \Pr(z \sim P_{\tau} = r) \sum_{m \in M} q_m \left[\tilde{b}_{\tau,v,r} - y_{\tau,v,r,m} + \sum_u y_{\tau,u,r,m} \mathbf{1}[d_r = v] \right] \quad \forall \tau, v \quad (\text{FB})$$

The outer sum takes the expectation over which request r is drawn; the inner sum takes the expectation over the collapsed scenario m . Inside the brackets: $\tilde{b}_{\tau,v,r}$ is how many taxis were at v after Stage 1; we subtract those dispatched from v and add those that arrive at v after completing service (i.e. taxis assigned from any node u to a request whose destination d_r is v). This constraint poses a problem to the model as enforcing $b_{\tau+1,v}$ integral will likely not be feasible for fractional probabilities.

Variable Bounds:

$$y_{\tau,v,r,m} \in \mathbb{N} \quad \forall \tau, v, r, m \quad (\text{D1})$$

$$f_{\tau,u,v,r} \in \mathbb{N} \quad \forall \tau, u, v, r \quad (\text{D2})$$

$$b_{\tau,v} \in \mathbb{N} \quad \forall \tau, v \quad (\text{D3})$$

Assumptions Made

1. We assume all elevators have the same capacity, C
2. We assume that a request is of the form (s_t, d_t) ; we receive the origin and destination of the request at stage 1
3. We assume that we *do not* receive the size of the request until the first elevator arrives to s_t
4. We know ahead of time the distributions of arrival/destination locations and sizes for each time step (not necessarily iid)

5 Problems with the Full Two-Stage Capacity-Constrained Model

There are two main issues with the full formulation as it stands. First, solving this model before the events begin to occur is essentially an offline model that does not take advantage of information as it is learned. The model must take into account all possible runouts of arrival locations and sizes even after the true information has been revealed, making the model potentially less effective than one that can continually incorporate the current state of the system. Also, as discussed earlier, the flow balance constraint poses a significant challenge. When all repositioning and current elevator count variables are integral, while probabilities of different situations are fractional, it would be in the best case very difficult to satisfy the flow balance constraint while maintaining integrality and, more likely, impossible. Furthermore, the results from the literature that involve rounding fractional solutions to integral ones while maintaining the expected cost do not apply under these circumstances because of the extra expectation taken over the size of the demand. These issues forced us to pivot to a different strategy: Model Predictive Control.

6 Model Predictive Control (MPC) Formulation

Motivation for MPC

The two-stage stochastic formulation provides a natural way to model uncertainty in demand through recourse decisions. However, it suffers from a key structural limitation: the flow balance constraint computes the *expected* next-state configuration rather than a realizable one. In particular, the update

$$b_{\tau+1,v} = \sum_r \Pr(z \sim P_\tau = r) \sum_{m \in M} q_m(\dots)$$

typically produces fractional values of $b_{\tau+1,v}$, which do not correspond to any physical configuration of taxis. As a result, future constraints are enforced on states that cannot actually occur along any sample path. Additionally, the two-stage formulation commits to an entire trajectory of decisions before observing realizations of future requests and group sizes, preventing the model from adapting to new information.

To address these issues, we transition to a **Model Predictive Control (MPC)** framework. MPC is a receding-horizon control method in which a finite-horizon optimization problem is solved at each time step, only the first-stage decisions are implemented, and the problem is re-solved after observing updated system information. This repeated re-optimization provides an implicit feedback mechanism that allows the controller to adapt dynamically to uncertainty while still planning ahead.

MPC Framework

At each time step τ , the system observes the true configuration of taxis $\hat{b}_{\tau,v}$ and the realized request r^* . Rather than optimizing over the entire time horizon, we solve a finite-horizon optimization problem over periods $\tau, \dots, \tau + L$, where L is a lookahead parameter. The optimization includes a detailed two-stage model for the current period and a relaxed lookahead model for future periods.

Importantly, only the decisions for the current time step are implemented. After execution, the system observes the realized group size and updates the taxi configuration deterministically. The optimization is then repeated at the next time step using the new observed state.

MPC Algorithm

Algorithm 1 MPC for the Capacity-Constrained k -Taxi Problem

- 1: **Initialize:** Set $\hat{b}_{1,v}$ to the initial taxi configuration.
- 2: **for** $\tau = 1, 2, \dots, T$ **do**
- 3: **Observe** realized request r^* drawn from P_τ .
- 4: **Solve** the two-stage MIP with:
 - current state $\hat{b}_{\tau,v}$ (fixed, not a variable),
 - observed request r^* ,
 - lookahead cost $V_{\tau+1}$ from solving lookahead LP initialized at the expected termination pattern
- 5: **Execute Stage 1:** apply repositioning $f_{\tau,u,v}^*$. Compute $\tilde{b}_{\tau,v} = \hat{b}_{\tau,v} + \sum_u f_{\tau,u,v}^* - \sum_u f_{\tau,v,u}^*$.
- 6: **Observe** realized group size n_{r^*} ; determine actual scenario $m^* = \lceil n_{r^*}/C \rceil$.
- 7: **Execute Stage 2:** apply assignment y_{τ,v,m^*}^* .
- 8: **Update** actual state for next period:

$$\hat{b}_{\tau+1,v} = \tilde{b}_{\tau,v} - y_{\tau,v,m^*}^* + \sum_u y_{\tau,u,m^*}^* \mathbf{1}[d_{r^*} = v].$$

9: **end for**

Lookahead LP

We treat the problem essentially like a two-stage model where the current time period decisions are the first stage and all later decisions (within the lookahead window) are second stage. Since it is impossible to compute the second stage value for integral variables, we relax all of them and solve a simple LP. For each potential decision for the current time-step, we compute the expected elevator distribution at the start of the next time-step, $a_{\tau,v}$:

$$a_{\tau,v} = \sum_{m \in M} q_m \left[\tilde{b}_{\tau,v} - y_{\tau,v,m} + \sum_u y_{\tau,u,m} \mathbf{1}[d_{r^*} = v] \right]. \quad (1)$$

We then solve the following LP to estimate the cost that will be incurred from time $\tau + 1$ and on, $V_{\tau+1}$:

$$\begin{aligned}
 V_{\tau+1}(a_\tau) = \min \quad & \sum_{t=\tau+1}^{\tau+L} \left[\sum_{u,v} f_{t,u,v} d(u,v) + \sum_v \sum_r \Pr(z \sim P_t = r) x_{t,v,r} d(v, s_r) \right] \\
 \text{s.t.} \quad & b_{\tau+1,v} = a_{\tau,v} & \forall v \\
 & b_{t+1,v} = b_{t,v} + \sum_u f_{t,u,v} - \sum_u f_{t,v,u} \\
 & \quad - \sum_r \Pr(z \sim P_t = r) x_{t,v,r} + \sum_r \Pr(z \sim P_t = r) \sum_u x_{t,u,r} \mathbf{1}[d_r = v] & \forall t, v \\
 & \sum_v x_{t,v,r} \geq \bar{m} & \forall t, r \\
 & x_{t,v,r} \leq b_{t,v} & \forall t, v, r \\
 & \sum_v b_{t,v} \leq k & \forall t \\
 & f_{t,u,v}, x_{t,v,r}, b_{t,v} \geq 0 & \forall t, u, v, r
 \end{aligned}$$

Explanation and bias. The lookahead LP is optimistic in two ways. First, it allows for fractional flow balance, so future stages do not have to represent feasible arrangements of elevators. Second, in order to keep the problem small and quick to solve, we compute $\bar{m}_t$ for each time step which is the expected number of taxis required based on the arrival size distribution. This simplifies the model significantly as we do not need to consider all possible request sizes and compute expectations over them. However, this is a simplification and thus creates some bias in the model. We justify this by arguing that the speed-up in solve time is worth the approximation error, especially since this value is used only used to inform decisions and is constantly updated as more information is received.

Key Advantages of MPC

The MPC framework resolves the primary limitations of the two-stage stochastic model:

- **Realizable states:** The system state is updated using the *realized* scenario rather than an expectation, ensuring that all configurations remain feasible.
- **Adaptivity:** Decisions are re-optimized at every time step using newly observed information, allowing the model to react dynamically to realized demand rather than committing to a fixed plan.
- **Tractability:** By limiting optimization to a finite lookahead horizon and relaxing future variables, MPC avoids the computational complexity of solving a full multi-stage stochastic program, solving the problem within real world time constraints.

7 Results

We evaluate the performance of the Model Predictive Control (MPC) policy against two benchmarks: a greedy baseline and the offline optimal solution. The greedy policy never repositions elevators proactively and always dispatches the closest available elevators to serve each request. The offline optimal solution is computed using a deterministic integer program with full knowledge of all realized requests and group sizes.

Models

1. **Lazy/Greedy:** Never reposition taxis in stage one, always send only the closest taxi to explore s_t , then fulfill the demand using the closest taxis. This is most similar to how real world elevators work.
2. **Model Predictive Control:** As described above, solve a two-stage model at each time-step based on a set lookahead window to determine immediate routing.
3. **Offline Optimal** This model receives all of the requests and their sizes ahead of time. It then plots the best possible route fulfilling all demands. Note that it this algorithm is never charged the λ stage 2 penalty because it can always arrange the correct number of taxis in stage 1.

Experimental Setup

We test the algorithms across 15 scenarios designed to capture a wide range of real world demand patterns, including uniform arrivals, lobby-heavy demand, long-distance trips, alternating traffic patterns, high-variance groups, and adversarial cases. Each scenario is evaluated over 50 independent replications. The number of floors ranges from 4 to 100, and the number of elevators varies from 2 to 10.

Test	Floors	Elevators	Demand Pattern
1	4	2	Three fixed requests
2	10	4	Top-to-lobby weighted demand
3	8	3	Rotating patterns with high-variance large groups
4	8	3	Uniform demand with groups concentrated at size 3
5	10	4	Six alternating demand patterns
6	15	4	Morning up-rush followed by evening reversal
7	20	5	Fully uniform demand
8	20	5	Traffic concentrated around middle floors
9	20	6	Demand flips between bottom and top floors each period
10	25	5	Uniform demand with groups up to size 8
11	25	6	Strong lobby-biased demand
12	15	5	Five patterns cycled three times over 15 steps
13	100	10	Lobby-only demand
14	25	6	Requests only from floors 0 and 24 to floor 12
15	30	6	Requests only between floors 0 and 29 with large groups of 7–9 passengers

Table 1: Summary of test scenarios used to evaluate MPC.

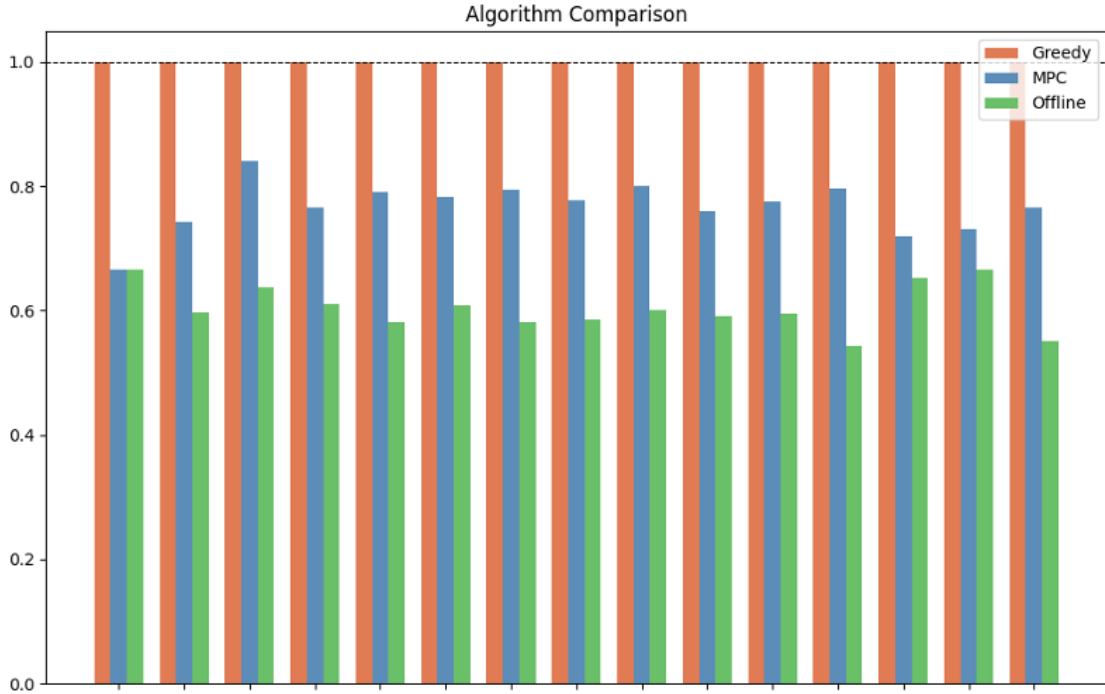


Figure 1: Comparison of greedy, MPC, and offline optimal performance across the synthetic test suite.

Performance Comparison

Across the test suite, MPC consistently performs close to the offline optimal solution. In most scenarios, the ratio of MPC cost to offline optimal cost remains low, showing that MPC achieves near-optimal performance despite operating online without full knowledge of future requests.

Compared to the greedy baseline, MPC provides a substantial improvement. Since the greedy policy only reacts to the current request, it often leaves elevators poorly positioned for future demand. MPC avoids this issue by using the lookahead model to reposition elevators proactively.

Insights

- MPC significantly outperforms greedy dispatching by anticipating future demand instead of only reacting to the current request.
- The largest performance gaps between MPC and the offline optimal occur in highly variable or adversarial scenarios, where future demand is harder to predict.
- MPC performs especially well in structured settings, such as lobby-heavy demand, alternating traffic patterns, and repeated demand cycles.
- The offline optimal remains a useful lower bound because it has full knowledge of future requests, while MPC must make decisions online.
- Overall, the results suggest that proactive repositioning is valuable for reducing deadhead distance in capacity-constrained elevator dispatching.

Possibility for a Regret Bound

The MPC algorithm we designed fits into the category of re-solving heuristics. While there is not much literature on bounding the regret of algorithms exactly like ours, the closest match we could find is in the network revenue management problem. Papers by Bumpensanti and Wang [8] and Wang and Wang [9] show that re-solving heuristics can achieve $O(1)$ regret against a hindsight optimal solution, but only under conditions that do not hold in our setting: either a non-degeneracy assumption on the fluid LP, or a specialized infrequent re-solving schedule with thresholding. Our model has high probability of degenerate LP lookaheads due to the symmetry between taxis and the line metric, and the thresholding techniques don't carry over to spatial assignment. Without these tools, the same authors prove a matching $\Omega(\sqrt{T})$ lower bound on the regret of frequent re-solving, so we conjecture a bound of $O(\sqrt{T})$ on the regret of our MPC, though we do not have enough time as of now to prove this result.

Conclusion

Our results suggest that MPC is a practical and effective approach for the capacity-constrained k-taxi, achieving less than $\frac{3}{2}$ times the offline movement cost in all but one experiment. For situations with known distributions and a relatively short time horizon, the cost of the MPC algorithm is quite good in relation to the offline optimal and naive approaches. As the number of elevators and time steps increases, we predict the solve time increases quickly, rendering the algorithm less practical for real-world settings. While it does not achieve the theoretical optimality of the offline solution and proving a bound on regret may be difficult, it provides a strong balance between computational tractability and solution quality. We look forward to the possibility of finding a theoretical guarantee or extending the algorithm to further improve our results.

8 References

References

- [1] M. S. Manasse, L. A. McGeoch, and D. D. Sleator, “Competitive Algorithms for Server Problems,” *Journal of Algorithms*, vol. 11, no. 2, pp. 208–230, Jun. 1990. doi:10.1016/0196-6774(90)90003-W.
- [2] S. Dehghani, S. Ehsani, M. Hajiaghayi, V. Liaghat, and S. Seddighin, “Stochastic k -Server: How Should Uber Work?,” in *44th International Colloquium on Automata, Languages, and Programming (ICALP 2017)*, LIPIcs, vol. 80, Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017, pp. 126:1–126:14. doi:10.4230/LIPIcs.ICALP.2017.126.
- [3] C. Coester and E. Koutsoupias, “The Online k -Taxi Problem,” in *Proc. 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, Phoenix, AZ, USA, 2019, pp. 1136–1147. doi:10.1145/3313276.3316370.
- [4] C. Coester and T.-Y. Poon, “Online 3-Taxi on General Metrics,” in *Proc. 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, Vancouver, Canada, 2026, pp. 6659–6673. doi:10.1137/1.9781611978971.238. arXiv:2510.25861.
- [5] D. Nikovski and M. Brand, “Decision-Theoretic Group Elevator Scheduling,” in *Proc. 13th Int. Conf. on Automated Planning and Scheduling (ICAPS)*, Trento, Italy, 2003, pp. 133–142.
- [6] I. L. Gørtz, V. Nagarajan, and R. Saket, “Stochastic Vehicle Routing with Recourse,” in *Automata, Languages, and Programming: 39th Int. Colloquium (ICALP 2012)*, LNCS, vol. 7391, A. Czumaj, K. Mehlhorn, A. Pitts, and R. Wattenhofer, Eds. Berlin: Springer, 2012, pp. 411–423. doi:10.1007/978-3-642-31594-7_35. arXiv:1202.5797.
- [7] Y. Wang and S. Boyd, “Fast Model Predictive Control Using Online Optimization,” *IEEE Trans. Control Syst. Technol.*, vol. 18, no. 2, pp. 267–278, Mar. 2010. doi:10.1109/TCST.2009.2017934.
- [8] P. Bumpensanti and H. Wang, “A Re-Solving Heuristic with Uniformly Bounded Loss for Network Revenue Management,” *Management Sci.*, vol. 66, no. 7, pp. 2993–3009, Jul. 2020. doi:10.1287/mnsc.2019.3365.
- [9] Y. Wang and H. Wang, “Constant Regret Resolving Heuristics for Price-Based Revenue Management,” *Oper. Res.*, vol. 70, no. 6, pp. 3538–3557, Nov.–Dec. 2022. doi:10.1287/opre.2021.2219.